❑ Exercice 1 : *Intersection*

En OCaml, on représente une partie de $\mathbb{N}$ par une liste *triée* d'entiers, par exemple $[9; 15; 22; 28]$.

1. Écrire une fonction `intersection : int list -> int list -> int list` calculant l'intersection de deux parties (le résultat doit être trié).

```
let rec intersection l1 l2 =
  match l1, l2 with
  | [], _ | _, [] -> []
  | h1 :: t1, h2 :: t2 ->
    if h1 = h2 then
      h1 :: intersection t1 t2
    else if h1 < h2 then
      intersection t1 (h2 :: t2)
    else
```

> On exploite le fait que les deux listes sont triées : si les deux têtes sont égales, l'élément appartient à l'intersection. Sinon, on avance dans la liste ayant la plus petite tête.

2. Prouver la terminaison de `intersection`.

> Soit $V(l_1, l_2) = \text{longueur}(l_1) + \text{longueur}(l_2)$.
> La quantité $V(l_1, l_2)$ est à valeurs entières positives. À chaque appel récursif :
> — si $h_1 = h_2$, on appelle `intersection t1 t2` : la somme des longueurs diminue de 2 ;
> — si $h_1 < h_2$, on appelle `intersection t1 (h2::t2)` : la longueur de l_1 diminue de 1 et celle de l_2 est inchangée, donc la somme diminue de 1 ;
> — si $h_1 > h_2$, on appelle `intersection (h1::t1) t2` : la somme des longueurs diminue de 1.
>
> Dans tous les cas, $V \in \mathbb{N}$ décroît strictement à chaque appel récursif. C'est donc un variant qui assure la terminaison.

❑ Exercice 2 : *Correction*

1. Écrire en C une fonction de signature `int somme(int tab[], int taille)` renvoyant la somme des éléments d'un tableau.

```
int somme(int tab[], int taille) {
    int s = 0;
    for (int i = 0; i < taille; i++) {
        s += tab[i];
    }
    return s;
}
```

2. Prouver la correction de votre fonction.

Montrons la correction à l'aide d'un invariant de boucle.
Pour tout $i \in [\![0; \text{taille}]\!]$, on note $\mathcal{I}(i)$ la propriété : « Au début du tour d'indice i, la variable `s` contient la somme des i premiers éléments du tableau, c'est-à-dire $s = \sum_{k=0}^{i-1} \texttt{tab}[k]$ » (avec la convention qu'une somme vide vaut 0).

— **Initialisation** ($i = 0$) : à l'entrée dans la boucle, $s = 0$, ce qui correspond bien à la somme vide des 0 premiers éléments. $\mathcal{I}(0)$ est donc vérifiée.

— **Conservation** : supposons $\mathcal{I}(i)$ vraie pour un entier $0 \leqslant i <$ taille. Lors de l'itération, l'instruction $s \leftarrow s + \texttt{tab}[i]$ met à jour s, puis l'indice passe à $i+1$. La nouvelle valeur de s est $\left(\sum_{k=0}^{i-1} \texttt{tab}[k]\right) + \texttt{tab}[i] = \sum_{k=0}^{i} \texttt{tab}[k]$. La propriété $\mathcal{I}(i+1)$ est donc vérifiée.

— **Terminaison** : la boucle se termine lorsque $i =$ taille. L'invariant $\mathcal{I}$(taille) garantit alors que $s = \sum_{k=0}^{\text{taille}-1} \texttt{tab}[k]$, ce qui correspond exactement à la somme de tous les éléments du tableau. La valeur renvoyée est donc correcte.

❑ Exercice 3 : *Notation $\mathcal{O}$*

1. Déterminer un grand $\mathcal{O}$ des suites de terme général :
   a) $2023n^2$ b) $n^2 + 10^9 n$ c) $3n + 7\log n$
   d) $2^{n+7} + n^{10}$ e) $\sqrt{19n^2+3}$ f) $\log(3n) + \log(n)$

a) $2023n^2 = \mathcal{O}(n^2)$ car $2023n^2 \leqslant 2023n^2$ pour tout $n \geqslant 1$.
b) $n^2 + 10^9 n = \mathcal{O}(n^2)$ car pour tout $n \geqslant 10^9$, $n^2 + 10^9 n \leqslant 2n^2$.
c) $3n + 7\log n = \mathcal{O}(n)$ car $\log n \leqslant n$, donc $3n + 7\log n \leqslant 10n$.
d) $2^{n+7} + n^{10} = 128 \times 2^n + n^{10} = \mathcal{O}(2^n)$ car par croissances comparées, $n^{10} = \mathcal{O}(2^n)$.
e) $\sqrt{19n^2+3} = \mathcal{O}(n)$ car pour $n \geqslant 1$, $19n^2 + 3 \leqslant 20n^2$, d'où $\sqrt{19n^2+3} \leqslant \sqrt{20}\, n$.
f) $\log(3n) + \log(n) = \log 3 + 2\log n = \mathcal{O}(\log n)$.

2. Montrer que si $u_n = \mathcal{O}(v_n)$ et $v_n = \mathcal{O}(w_n)$, alors $u_n = \mathcal{O}(w_n)$.

Par définition, il existe des réels $K_1, K_2 > 0$ et des entiers $N_1, N_2 \in \mathbb{N}$ tels que pour tout $n \geqslant N_1$, $u_n \leqslant K_1 v_n$ et pour tout $n \geqslant N_2$, $v_n \leqslant K_2 w_n$. En posant $N = \max(N_1, N_2)$ et $K = K_1 K_2 > 0$, on a pour tout $n \geqslant N$:

$$u_n \leqslant K_1 v_n \leqslant K_1 (K_2 w_n) = K w_n$$

ce qui prouve que $u_n = \mathcal{O}(w_n)$.

3. Montrer que $\mathcal{O}(u_n) + \mathcal{O}(v_n) = \mathcal{O}(\max(u_n, v_n))$.

Soit $x_n = \mathcal{O}(u_n)$ et $y_n = \mathcal{O}(v_n)$. Il existe des constantes $K_1, K_2 > 0$ et des rangs N_1, N_2 tels que pour tout $n \geqslant \max(N_1, N_2)$:

$$x_n + y_n \leqslant K_1 u_n + K_2 v_n \leqslant (K_1 + K_2) \max(u_n, v_n)$$

Toute somme d'un élément de $\mathcal{O}(u_n)$ et d'un élément de $\mathcal{O}(v_n)$ est donc dominée par $\max(u_n, v_n)$, d'où $\mathcal{O}(u_n) + \mathcal{O}(v_n) = \mathcal{O}(\max(u_n, v_n))$.

4. Montrer que si $u_n = \mathcal{O}(a_n)$ et $v_n = \mathcal{O}(b_n)$, alors $u_n v_n = \mathcal{O}(a_n b_n)$.

Il existe $K_1, K_2 > 0$ et $N_1, N_2 \in \mathbb{N}$ tels que pour tout $n \geqslant \max(N_1, N_2)$, $u_n \leqslant K_1 a_n$ et $v_n \leqslant K_2 b_n$. Comme les suites sont à termes strictement positifs :

$$u_n v_n \leqslant (K_1 a_n)(K_2 b_n) = (K_1 K_2)(a_n b_n)$$

ce qui établit que $u_n v_n = \mathcal{O}(a_n b_n)$.

5. Déterminer un grand $\mathcal{O}$ (le « meilleur » possible) des expressions suivantes :
   a) $\mathcal{O}(n^4) + \mathcal{O}(n^2)$ b) $\mathcal{O}(n^5) + \mathcal{O}(n^5)$ c) $\mathcal{O}(n^3) + \mathcal{O}(\log(n))$
   d) $\mathcal{O}(n^4) \times \mathcal{O}(n^3)$ e) $\mathcal{O}(n^4) \times \mathcal{O}(\sqrt{n})$ f) $\mathcal{O}(n^2) \times \mathcal{O}(\log(n))$

a) $\mathcal{O}(n^4) + \mathcal{O}(n^2) = \mathcal{O}(\max(n^4, n^2)) = \mathcal{O}(n^4)$.
b) $\mathcal{O}(n^5) + \mathcal{O}(n^5) = \mathcal{O}(n^5)$.
c) $\mathcal{O}(n^3) + \mathcal{O}(\log n) = \mathcal{O}(n^3)$.
d) $\mathcal{O}(n^4) \times \mathcal{O}(n^3) = \mathcal{O}(n^{4+3}) = \mathcal{O}(n^7)$.
e) $\mathcal{O}(n^4) \times \mathcal{O}(\sqrt{n}) = \mathcal{O}(n^{4+1/2}) = \mathcal{O}(n^{9/2})$.
f) $\mathcal{O}(n^2) \times \mathcal{O}(\log n) = \mathcal{O}(n^2 \log n)$.

❑ Exercice 4 : *Multiplier en additionnant*

```c
int mult(int n, int p){
    int prod = 0;
    while (p>0){
        prod = prod + n;
        p = p -1;}
    return prod;}
```

1. En supposant $p > 0$, montrer la terminaison de la fonction `mult`.

La variable p est à valeurs entières. Comme $p > 0$ initialement et que p décroît strictement de 1 à chaque itération ($p \leftarrow p - 1$), p est un variant de boucle qui atteint 0 en exactement p tours, ce qui garantit la terminaison.

2. Prouver que cette fonction renvoie bien $p \times n$.

Notons p_0 la valeur initiale de p. Considérons l'invariant : $\texttt{prod} + p \times n = p_0 \times n$.
- **Entrée** : $\texttt{prod} = 0$, donc $0 + p_0 \times n = p_0 \times n$, vérifié.
- **Conservation** : lors d'un tour, $\texttt{prod}' = \texttt{prod} + n$ et $p' = p - 1$, donc $\texttt{prod}' + p'n = (\texttt{prod} + n) + (p-1)n = \texttt{prod} + pn = p_0 n$. L'invariant est conservé.
- **Sortie** : la boucle s'arrête quand $p = 0$. Alors $\texttt{prod} + 0 = p_0 n \implies \texttt{prod} = p_0 n$. La fonction renvoie bien le produit attendu.

3. Déterminer sa complexité.

La boucle effectue exactement p itérations, chacune contenant un nombre constant d'opérations élémentaires ($\mathcal{O}(1)$). La complexité temporelle est donc en $\mathcal{O}(p)$ (linéaire en p, indépendante de n).

❑ Exercice 5 : *Vérification du tri*

1. Écrire un algorithme permettant de vérifier qu'un tableau est trié par ordre croissant.

```c
bool est_trie(int tab[], int n) {
    for (int i = 0; i < n - 1; i++) {
        if (tab[i] > tab[i + 1]) {
            return false;
        }
    }
    return true;
}
```

2. En proposer une implémentation en OCaml permettant de vérifier qu'une liste est triée.

```
let rec est_triee = function
  | [] | [_] -> true
  | x :: (y :: _ as t) -> x <= y && est_triee t
```

3. Prouver que votre algorithme est correct.

> • Si le tableau n'est pas trié, il existe au moins un indice i tel que $t[i] > t[i+1]$. Dès la première occurrence d'une telle paire, l'instruction `return false` est exécutée, ce qui est correct.
> • Si le tableau est trié, la condition $t[i] > t[i+1]$ n'est jamais vérifiée pour tout $i \in [\![0; n-2]\!]$. La boucle s'exécute jusqu'au bout et l'instruction `return true` est atteinte, ce qui prouve la correction.

4. Déterminer sa complexité.

> • **Meilleur des cas** : $\mathcal{O}(1)$ si les deux premiers éléments ne sont pas triés ($t[0] > t[1]$), la fonction s'arrête dès la première comparaison.
> • **Pire des cas** : $\mathcal{O}(n)$ si le tableau est trié (ou si la seule inversion se situe à la fin), la boucle effectue $n-1$ comparaisons.

❑ **Exercice 6** : *Exponentiation rapide*

On rappelle la fonction d'exponentiation rapide dans sa version itérative :

```
float expo(float a, int n){
    float cp = a;
    float res = 1;
    while (n!=0){
        if (n%2==1){
            res = res*cp;}
        cp = cp*cp;
        n=n/2;}
    return res;}
```

1. Prouver que cet algorithme termine.

> La variable n est un entier positif au départ. À chaque tour de boucle, n est remplacé par sa division entière par 2 ($n \leftarrow \lfloor n/2 \rfloor$). Ainsi, n décroît strictement pour tout $n \geqslant 1$ et prend ses valeurs dans $\mathbb{N}$. C'est un variant de boucle qui garantit la terminaison en au plus $\lfloor \log_2 n_0 \rfloor + 1$ itérations.

2. Prouver qu'il est correct.
   ⊗ En notant n_0 la valeur initiale de n, on pourra considérer l'invariant suivant : $\texttt{res} \times \texttt{cp}^n = a^{n_0}$.

Montrons que $\texttt{res} \times \texttt{cp}^n = a^{n_0}$ est un invariant de boucle.

— **Initialisation** : à l'entrée dans la boucle, $\texttt{res} = 1$, $\texttt{cp} = a$ et $n = n_0$, donc $\texttt{res} \times \texttt{cp}^n = 1 \times a^{n_0} = a^{n_0}$.

— **Conservation** : soient $\texttt{res}, \texttt{cp}, n$ les valeurs au début d'un tour et $\texttt{res}', \texttt{cp}', n'$ les valeurs après le tour. On a $\texttt{cp}' = \texttt{cp}^2$ et $n' = \lfloor n/2 \rfloor$.

— Si n est pair ($n = 2k$) : $\texttt{res}' = \texttt{res}$ et $n' = k$, donc :

$$\texttt{res}' \times (\texttt{cp}')^{n'} = \texttt{res} \times (\texttt{cp}^2)^k = \texttt{res} \times \texttt{cp}^{2k} = \texttt{res} \times \texttt{cp}^n = a^{n_0}$$

— Si n est impair ($n = 2k+1$) : $\texttt{res}' = \texttt{res} \times \texttt{cp}$ et $n' = k$, donc :

$$\texttt{res}' \times (\texttt{cp}')^{n'} = (\texttt{res} \times \texttt{cp}) \times (\texttt{cp}^2)^k = \texttt{res} \times \texttt{cp}^{2k+1} = \texttt{res} \times \texttt{cp}^n = a^{n_0}$$

L'invariant est conservé dans les deux cas.

— **Sortie** : la boucle se termine quand $n = 0$. Alors $\texttt{res} \times \texttt{cp}^0 = a^{n_0} \implies \texttt{res} = a^{n_0}$. L'algorithme renvoie bien a^{n_0}.

3. Donner sa complexité.

À chaque passage dans la boucle, la valeur de n est divisée par 2. Le nombre d'itérations est donc $\lfloor \log_2 n \rfloor + 1$. Chaque itération effectuant un nombre borné d'opérations élémentaires ($\mathcal{O}(1)$), la complexité temporelle est en $\mathcal{O}(\log n)$.

❑ **Exercice 7** : *Retour sur la multiplication*

On donne la fonction suivante :

```
1  int multiplie(int n, int p){
2      int prod = 0;
3      while (n>0){
4          if (n%2==1) {
5              prod = prod+p;}
6          n= n / 2;
7          p = p*2;}
8      return prod;}
```

1. Vérifier à la main, sur deux entiers naturels de votre choix, que cette fonction est conforme à sa spécification.

Pour $n = 12$ et $p = 30$:

— Début : $\texttt{prod} = 0, n = 12, p = 30$.

— $n = 12$ (pair) : $\texttt{prod} = 0, n = 6, p = 60$.

— $n = 6$ (pair) : $\texttt{prod} = 0, n = 3, p = 120$.

— $n = 3$ (impair) : $\texttt{prod} = 0 + 120 = 120, n = 1, p = 240$.

— $n = 1$ (impair) : $\texttt{prod} = 120 + 240 = 360, n = 0, p = 480$.

La boucle s'arrête ($n = 0$) et renvoie $\texttt{prod} = 360$, ce qui correspond bien à $12 \times 30 = 360$.

2. Montrer la terminaison de cette fonction.

La variable $n \in \mathbb{N}$ est divisée par 2 à chaque itération ($n \leftarrow \lfloor n/2 \rfloor$). C'est un variant de boucle entier qui décroît strictement tant que $n > 0$ et s'annule en au plus $\lfloor \log_2 n_0 \rfloor + 1$ itérations, ce qui prouve la terminaison.

3. Montrer que cette fonction est bien conforme à sa spécification.

Montrons que $\texttt{prod} + n \times p = n_0 \times p_0$ est un invariant de boucle.

— **Initialisation** : $\texttt{prod} = 0$, donc $0 + n_0 p_0 = n_0 p_0$, vérifié.

— **Conservation** : si n est pair ($n = 2k$), $\texttt{prod}' = \texttt{prod}, n' = k, p' = 2p$, d'où :

$$\texttt{prod}' + n'p' = \texttt{prod} + k(2p) = \texttt{prod} + np = n_0 p_0$$

Si n est impair ($n = 2k + 1$), $\texttt{prod}' = \texttt{prod} + p, n' = k, p' = 2p$, d'où :

$$\texttt{prod}' + n'p' = \texttt{prod} + p + 2kp = \texttt{prod} + (2k + 1)p = \texttt{prod} + np = n_0 p_0$$

— **Sortie** : quand $n = 0$, $\texttt{prod} + 0 = n_0 p_0 \implies \texttt{prod} = n_0 p_0$. La fonction est correcte.

4. Déterminer sa complexité.

La boucle effectue $\lfloor \log_2 n \rfloor + 1$ passages, chacun coûtant un nombre borné d'opérations élémentaires. La complexité temporelle est donc en $\mathcal{O}(\log n)$ (algorithme de la multiplication russe).

❑ Exercice 8 : *Tri à bulles*

1. Rappeler le principe du tri à bulles.

Le principe est de parcourir le tableau de gauche à droite en comparant chaque élément $t[i]$ à son successeur $t[i + 1]$ et en les échangeant s'ils sont dans le mauvais ordre ($t[i] > t[i + 1]$). À la fin du premier passage, le plus grand élément est remonté en dernière position. On réitère le processus sur le sous-tableau restant.

2. En écrire une implémentation en C, dans laquelle on vérifie à chaque passage qu'au moins une inversion a été effectuée. Si tel n'est pas le cas, on termine immédiatement l'algorithme puisque cela signifie que les éléments sont triés.

```c
void tri_bulles(int t[], int n) {
    bool modif = true;
    int passage = 0;
    while (modif && passage < n - 1) {
        modif = false;
        for (int i = 0; i < n - 1 - passage; i++) {
            if (t[i] > t[i + 1]) {
                int tmp = t[i];
                t[i] = t[i + 1];
                t[i + 1] = tmp;
                modif = true;
            }
        }
        passage++;
    }
}
```

3. Montrer la terminaison de cette fonction.

La quantité $(n - 1 - \texttt{passage})$ est un variant de boucle : elle prend des valeurs entières, est positive au départ, et décroît strictement de 1 à chaque passage dans la boucle `while`. De plus, la boucle peut s'arrêter prématurément si `modif` reste `false`. La terminaison est donc assurée.

4. Prouver qu'elle est correcte.
   🛟 On pourra exhiber un invariant qui donne le nombre d'éléments figurant à leur place finale après chaque itération.

   • **Invariant** : après k passages, les k plus grands éléments du tableau occupent les positions finales $t[n-k \dots n-1]$ dans l'ordre trié, et sont supérieurs ou égaux à tous les éléments de $t[0 \dots n-k-1]$.
   • Si un passage s'effectue sans aucun échange (`modif == false`), cela signifie que pour tout $i \in [\![0; n-2-\text{passage}]\!]$, $t[i] \leqslant t[i+1]$. Le sous-tableau restant étant trié et ses éléments étant inférieurs à la zone finale, le tableau complet est trié : l'arrêt anticipé est parfaitement correct.

5. Déterminer la complexité de cet algorithme dans le pire des cas et dans le meilleur des cas.

   • **Meilleur des cas** : si le tableau est déjà trié, le premier passage n'effectue aucun échange (`modif` reste `false`). L'algorithme s'arrête après $n-1$ comparaisons, d'où une complexité en $\mathcal{O}(n)$.
   • **Pire des cas** : si le tableau est trié par ordre décroissant, $n-1$ passages sont nécessaires, effectuant $\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \mathcal{O}(n^2)$ comparaisons et échanges.

❑ Exercice 9 : *Recherche dichotomique*

Algorithme : Recherche dichotomique

Entrées : $a \in \mathbb{N}$ et un tableau d'entiers t **trié** de longueur n
Sorties : Booléen indiquant si $a \in t$.

```
1  d ← 0
2  f ← n − 1
3  tant que f − d ⩾ 0 faire
4  |  m ← (f + d)/2
5  |  si t[m] = a alors
6  |  |  return Vrai
7  |  fin
8  |  si a < t[m] alors
9  |  |  f ← m − 1
10 |  fin
11 |  sinon
12 |  |  d ← m + 1
13 |  fin
14 fin
15 return Faux
```

1. Étudier la terminaison de cet algorithme.

   Considérons la quantité $V = f - d + 1$. À l'entrée dans la boucle, $V = n \geqslant 0$. Tant que la condition de boucle est vérifiée, on a $f - d \geqslant 0$. Soit $m = \lfloor (f+d)/2 \rfloor$.
   — Si $a < t[m]$, la nouvelle borne supérieure est $f' = m - 1$, donc $f' - d' + 1 = m - d = \lfloor \frac{f-d}{2} \rfloor < f - d + 1$.
   — Si $a > t[m]$, la nouvelle borne inférieure est $d' = m + 1$, donc $f' - d' + 1 = f - m = \lceil \frac{f-d}{2} \rceil < f - d + 1$.
   Dans les deux cas, $V \in \mathbb{N}$ décroît strictement à chaque itération. Il existe donc un moment où $V \leqslant 0$ (c'est-à-dire $f - d < 0$), ce qui prouve la terminaison.

2. Prouver que cet algorithme est totalement correct.

- Si l'algorithme renvoie `Vrai`, c'est qu'il a trouvé un indice m tel que $t[m] = a$, donc $a \in t$ (correction partielle).
- Si l'algorithme renvoie `Faux`, montrons l'invariant : « Si $a \in t$, alors $a \in t[d \dots f]$ ».
  - **Initialisation** : $d = 0$ et $f = n - 1$, le sous-tableau couvre tout t, la propriété est vraie.
  - **Conservation** : le tableau étant trié, si $a < t[m]$, alors pour tout $k \geqslant m$, $t[k] \geqslant t[m] > a$, donc a ne peut pas appartenir à $t[m \dots f]$. S'il est présent, il est dans $t[d \dots m-1]$. De même si $a > t[m]$, il est nécessairement dans $t[m+1 \dots f]$.
  - **Sortie** : si la boucle termine sans trouver a, on a $f < d$, donc l'intervalle $t[d \dots f]$ est vide. L'invariant implique donc que $a \notin t$, et le renvoi de `Faux` est correct.

L'algorithme se terminant toujours et étant partiellement correct, il est **totalement correct**.

3. Déterminer sa complexité.

À chaque itération, la longueur de l'intervalle de recherche est divisée par 2. Le nombre maximal d'itérations est donc $\lfloor \log_2 n \rfloor + 1$. Chaque itération contenant un nombre borné d'opérations élémentaires ($\mathcal{O}(1)$), la complexité temporelle est en $\mathcal{O}(\log n)$.

❑ Exercice 10 : *Nombre de chiffres d'un entier*

1. Écrire en C une version itérative d'une fonction donnant le nombre de chiffres d'un entier naturel.

```c
int nb_chiffres(int n) {
    if (n == 0) {
        return 1;
    }
    int count = 0;
    while (n > 0) {
        count++;
        n = n / 10;
    }
    return count;
}
```

2. Écrire une version récursive en OCaml.

```ocaml
let rec nb_chiffres n =
  if n < 10 then 1
  else 1 + nb_chiffres (n / 10)
```

3. Prouver la terminaison dans les deux cas.

Dans les deux cas, l'entier n est strictement divisé par 10 (division entière) à chaque étape. La quantité $n \in \mathbb{N}$ est un variant qui décroît strictement pour $n > 0$ et atteint 0 (ou une valeur < 10) en exactement $\lfloor \log_{10} n \rfloor + 1$ étapes pour tout $n \geqslant 1$. La terminaison est donc garantie.

4. Prouver la correction dans les deux cas.

Un entier naturel $n \geqslant 1$ s'écrit de manière unique en base 10 : $n = \sum_{k=0}^{c-1} d_k 10^k$ avec $d_{c-1} \neq 0$, où $c = \lfloor \log_{10} n \rfloor + 1$ est le nombre de chiffres. Chaque division entière par 10 élimine le chiffre de poids faible d_0. Le compteur s'incrémente de 1 à chaque division jusqu'à ce que tous les chiffres soient éliminés, ce qui donne exactement c. Pour $n = 0$, la condition initiale traite le cas particulier et renvoie 1 (le chiffre 0).

5. Déterminer la complexité dans les deux cas.

> Le nombre d'opérations est proportionnel au nombre d'itérations ou d'appels récursifs, c'est-à-dire $\mathcal{O}(\log_{10} n)$ (complexité logarithmique en la valeur de l'entier n, soit linéaire en la taille de son écriture décimale).

❑ Exercice 11 : *Majorité absolue*

On considère les résultats d'un vote sous la forme d'un tableau d'entiers positifs : lorsqu'on rencontre la valeur `i`, cela signifie que le candidat numéro `i` a obtenu un vote. Par exemple, si le tableau contient les valeurs `[2, 3, 1, 2, 2, 2, 4, 1, 1, 2]`, alors le candidat 1 a obtenu 3 voix, le candidat 2 a obtenu 5 voix, ... On cherche à concevoir un algorithme efficace permettant de déterminer (s'il existe) le candidat ayant obtenu la majorité absolue. Dans le cas où aucun candidat n'a la majorité absolue, l'algorithme doit renvoyer `-1`. On note C le nombre de candidats et N le nombre de votes.

1. Proposer un algorithme naïf permettant de résoudre ce problème et donner sa complexité.

> Un algorithme naïf consiste à parcourir le tableau pour chaque valeur candidate distincte afin de compter ses occurrences. Comme il y a au plus N candidats distincts et que chaque comptage prend un temps $\mathcal{O}(N)$, cet algorithme a une complexité temporelle en $\mathcal{O}(N^2)$ (pour une mémoire auxiliaire en $\mathcal{O}(1)$).

On donne ci-dessous **le début** de l'implémentation en C d'un algorithme proposé par R. Boyer et J. S. Moore :

```c
int majorite_absolue(int vote[], int size){
    int nb_votes = 0;
    int candidat = 0;
    for (int i = 0; i < size; i++){
        if (nb_votes == 0){
            candidat = vote[i];
            nb_votes = 1;}
        else{
            if (vote[i] == candidat){
                nb_votes += 1;}
            else{
                nb_votes -= 1;}
        }
    }
```

2. Prouver qu'à la fin de la boucle `for`, la variable `candidat` contient le numéro du seul candidat éventuellement majoritaire.

> L'algorithme de Boyer-Moore repose sur le principe d'élimination par paires d'éléments distincts : si deux votes pour des candidats distincts sont supprimés ensemble, le candidat majoritaire absolu (qui possède strictement plus de $N/2$ voix) conserve une majorité absolue stricte parmi les votes restants (puisqu'on retire au plus 1 vote pour lui et au moins 1 vote contre lui). La variable `nb_votes` compte l'excédent de voix du candidat courant. Lorsqu'elle atteint 0, cela signifie qu'un ensemble de votes a été apparié par paires distinctes et éliminé. À la sortie de la boucle, seul un candidat ayant la majorité absolue a pu être conservé dans `candidat`.

3. Compléter l'implémentation en ajoutant les lignes permettant de vérifier après la boucle `for` que ce candidat est effectivement majoritaire.

```
    int nbc = 0;
    for (int i = 0; i < size; i++)
    {
        if (vote[i] == candidat)
        {
            nbc += 1;
        }
    }
    if (nbc > size / 2)
    {
        return candidat;
    }
    else
    {
        return -1;
    }
```

4. Donner la complexité de cet algorithme.

L'algorithme effectue deux parcours consécutifs du tableau de taille N : la passe d'élimination de Boyer-Moore puis la vérification du candidat. Chaque parcours prend un temps $\mathcal{O}(N)$. La complexité temporelle est donc en $\mathcal{O}(N)$, et la mémoire auxiliaire est en $\mathcal{O}(1)$, ce qui est optimal.

❑ Exercice 12 : *Fonction 91*

La fonction 91, notée f_{91}, due à l'informaticien J. McCarthy est définie sur $\mathbb{N}$ par :

$$f_{91}(n) = \begin{cases} n - 10 & \text{si } n > 100, \\ f_{91}(f_{91}(n+11)) & \text{si } n \leqslant 100. \end{cases}$$

1. Calculer $f_{91}(101)$, $f_{91}(200)$, $f_{91}(2025)$.

- $f_{91}(101) = 101 - 10 = 91$.
- $f_{91}(200) = 200 - 10 = 190$.
- $f_{91}(2025) = 2025 - 10 = 2015$.

2. Écrire une fonction `f91` en OCaml calculant cette fonction.

```
let rec f91 n =
  if n > 100 then
    n - 10
  else
    f91 (f91 (n + 11))
```

3. Montrer que si $n \in [\![90; 100]\!]$ alors $f_{91}(n) = f_{91}(n+1)$. En déduire la valeur de f_{91} sur $[\![90; 100]\!]$.

Soit $n \in [\![90; 100]\!]$. Alors $n + 11 \in [\![101; 111]\!]$, donc $n + 11 > 100$.
Par définition, $f_{91}(n+11) = (n+11) - 10 = n + 1$.
On en déduit :

$$f_{91}(n) = f_{91}(f_{91}(n+11)) = f_{91}(n+1)$$

Ainsi, la fonction f_{91} est constante sur $[\![90; 101]\!]$:

$$f_{91}(90) = f_{91}(91) = \cdots = f_{91}(100) = f_{91}(101) = 91$$

4. Prouver que, pour tout $n \in [\![0; 101]\!]$, cette fonction termine et vaut 91. On pourra considérer l'entier $p \in \mathbb{N}^*$ tel que $90 \leqslant n + 11p \leqslant 100$.

Soit $n \in [\![0;101]\!]$. On raisonne par récurrence bien fondée sur le variant $V(n) = 101 - n \in \mathbb{N}$:

— Si $n \in [\![90;101]\!]$, on a déjà montré que la fonction termine et que $f_{91}(n) = 91$.

— Si $n < 90$, soit $p = \left\lceil \frac{90-n}{11} \right\rceil \geqslant 1$ le plus petit entier tel que $90 \leqslant n + 11p \leqslant 100$. On a $n + 11p \in [\![90;100]\!]$, donc $f_{91}(n + 11p) = 91$. Par une récurrence descendante, chaque appel interne $f_{91}(n + 11k)$ termine et se ramène finalement à 91.

La fonction termine donc pour tout $n \in [\![0;101]\!]$ et vaut identiquement 91.

❑ Exercice 13 : *Équations de récurrence*

Dans l'étude d'algorithmes récursifs, le nombre d'opérations élémentaires $T(n)$ pour une entrée de taille n satisfait souvent une équation de récurrence. On suppose dans tout l'exercice que $T(1) = 1$. Déterminer l'ordre de grandeur asymptotique en notation $\mathcal{O}$ de $T(n)$ dans chacun des cas suivants :

1. $T(n) = T(n-1) + n$ pour $n \geqslant 2$. Quel algorithme de tri usuel correspond à cette équation ?

Par sommation télescopique :

$$T(n) = T(1) + \sum_{k=2}^{n} k = 1 + \left(\frac{n(n+1)}{2} - 1 \right) = \frac{n(n+1)}{2} = \mathcal{O}(n^2)$$

Cette récurrence correspond par exemple au **tri par sélection** (ou insertion) récursif, où trouver le minimum parmi k éléments demande $k - 1 = \mathcal{O}(k)$ opérations, puis on trie récursivement les $k - 1$ éléments restants.

2. $T(n) = 2T(n-1) + 1$ pour $n \geqslant 2$. À quel problème algorithmique classique correspond cette récurrence ?

Posons $U(n) = T(n)+1$. Alors $U(n) = 2T(n-1)+2 = 2U(n-1)$. La suite $(U(n))$ est une suite géométrique de raison 2, avec $U(1) = T(1)+1 = 2$. D'où $U(n) = 2^n$, soit $T(n) = 2^n - 1 = \mathcal{O}(2^n)$. Cette équation correspond au problème des **tours de Hanoï** à n disques (on déplace $n-1$ disques, puis le plus grand disque, puis à nouveau les $n-1$ disques).

3. $T(n) = 2T(\lfloor n/2 \rfloor) + n$ pour $n \geqslant 2$. On pourra d'abord expliciter $T(2^k)$ pour tout $k \in \mathbb{N}$, puis en déduire la complexité dans le cas général.

Pour $n = 2^k$ avec $k = \log_2 n$, l'équation devient $T(2^k) = 2T(2^{k-1}) + 2^k$. En divisant par 2^k :

$$\frac{T(2^k)}{2^k} = \frac{T(2^{k-1})}{2^{k-1}} + 1$$

La suite $k \mapsto \frac{T(2^k)}{2^k}$ est une suite arithmétique de raison 1, avec $\frac{T(1)}{1} = 1$. D'où $\frac{T(2^k)}{2^k} = 1 + k = 1 + \log_2 n$, ce qui donne $T(2^k) = 2^k(1+k) = n(1 + \log_2 n) = \mathcal{O}(n \log n)$.
Par croissance de $T(n)$ et encadrement entre deux puissances de 2 ($2^k \leqslant n < 2^{k+1}$), on conclut que dans le cas général $T(n) = \mathcal{O}(n \log n)$. C'est la récurrence du **tri fusion** (*Mergesort*).

4. $T(n) = 3T(\lfloor n/2 \rfloor) + n$ pour $n \geqslant 2$. Même méthode que précédemment en étudiant $n = 2^k$ (on rappelle que cette récurrence apparaît dans l'algorithme de multiplication rapide d'entiers de Karatsuba).

Pour $n = 2^k$, posons $T(2^k) = 3T(2^{k-1}) + 2^k$. En divisant par 3^k :

$$\frac{T(2^k)}{3^k} = \frac{T(2^{k-1})}{3^{k-1}} + \left(\frac{2}{3}\right)^k$$

Par sommation télescopique :

$$\frac{T(2^k)}{3^k} = \frac{T(1)}{1} + \sum_{j=1}^{k} \left(\frac{2}{3}\right)^j \leqslant 1 + \sum_{j=1}^{+\infty} \left(\frac{2}{3}\right)^j = 1 + \frac{2/3}{1-2/3} = 3$$

Ainsi, $T(2^k) \leqslant 3 \times 3^k = 3 \times 3^{\log_2 n} = 3 \times n^{\log_2 3}$. Comme $\log_2 3 \approx 1{,}585$, on obtient une complexité en $\mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1{,}585})$.
Cette équation correspond à l'algorithme de **multiplication rapide d'entiers de Karatsuba**.

❑ **Exercice 14** : *Recherche simultanée du minimum et du maximum*
Soit t un tableau d'entiers de taille $n \geqslant 1$. On souhaite déterminer simultanément la valeur minimale et la valeur maximale de t.

1. Combien de comparaisons entre éléments du tableau effectue un algorithme naïf qui recherche successivement le minimum puis le maximum ?

   La recherche du minimum dans un tableau de taille n effectue $n-1$ comparaisons. La recherche du maximum effectue également $n-1$ comparaisons (ou $n-2$ si l'on exclut le minimum déjà identifié). L'approche naïve effectue donc au moins $2n-3$ comparaisons (et généralement $2n-2$).

2. On propose d'optimiser le nombre de comparaisons en traitant les éléments par paires :
   — Si n est impair, on initialise le minimum et le maximum courants à $t[0]$, et on examine les éléments restants deux par deux.
   — Si n est pair, on compare $t[0]$ et $t[1]$ pour initialiser le minimum et le maximum courants, et on examine les éléments restants deux par deux.
   — Pour chaque paire $\{t[i], t[i+1]\}$, on compare d'abord ces deux éléments entre eux, puis on compare le plus petit au minimum courant et le plus grand au maximum courant.

   Combien de comparaisons entre éléments du tableau sont effectuées pour chaque paire traitée ?

   Pour chaque paire $\{t[i], t[i+1]\}$:
   1. on compare $t[i]$ et $t[i+1]$ entre eux (1 comparaison) ;
   2. on compare le plus petit des deux au minimum courant (1 comparaison) ;
   3. on compare le plus grand des deux au maximum courant (1 comparaison).

   On effectue donc exactement **3 comparaisons** pour 2 éléments examinés (soit 1,5 comparaison par élément au lieu de 2).

3. En déduire le nombre total de comparaisons effectuées par cet algorithme en fonction de la parité de n. Montrer que, dans tous les cas, ce nombre vaut $\lceil \frac{3n}{2} \rceil - 2$.

   • **Si n est impair** ($n = 2k+1$) : 0 comparaison initiale ($\texttt{min} = \texttt{max} = t[0]$) et $k = \frac{n-1}{2}$ paires à traiter. Nombre total : $3 \times \frac{n-1}{2} = \frac{3n-3}{2}$.
   Or $\left\lceil \frac{3(2k+1)}{2} \right\rceil - 2 = \lceil 3k + 1{,}5 \rceil - 2 = (3k+2) - 2 = 3k = \frac{3n-3}{2}$.
   • **Si n est pair** ($n = 2k$) : 1 comparaison initiale entre $t[0]$ et $t[1]$, puis $k-1 = \frac{n-2}{2}$ paires à traiter. Nombre total : $1 + 3 \times \frac{n-2}{2} = \frac{3n-4}{2}$.
   Or $\left\lceil \frac{3(2k)}{2} \right\rceil - 2 = 3k - 2 = \frac{3n-4}{2}$.
   Dans les deux cas, la formule $\lceil \frac{3n}{2} \rceil - 2$ est rigoureusement exacte.

4. Écrire en C une fonction implémentant cet algorithme.

```c
void min_max(int t[], int n, int *pmin, int *pmax) {
    int i;
    if (n % 2 != 0) {
        *pmin = t[0];
        *pmax = t[0];
        i = 1;
    } else {
        if (t[0] < t[1]) {
            *pmin = t[0];
            *pmax = t[1];
        } else {
            *pmin = t[1];
            *pmax = t[0];
        }
        i = 2;
    }
    while (i < n - 1) {
        if (t[i] < t[i + 1]) {
            if (t[i] < *pmin) *pmin = t[i];
            if (t[i + 1] > *pmax) *pmax = t[i + 1];
        } else {
            if (t[i + 1] < *pmin) *pmin = t[i + 1];
            if (t[i] > *pmax) *pmax = t[i];
        }
        i += 2;
    }
```

5. Prouver formellement sa correction.

Preuve par invariant de boucle :
Soit $\mathcal{I}(i)$ la propriété : « Les variables pointées par `pmin` et `pmax` contiennent respectivement le minimum et le maximum des éléments du sous-tableau $t[0 \dots i-1]$ ».

— **Initialisation** : si n est impair, $i = 1$ et $t[0]$ est le seul élément examiné ; si n est pair, $i = 2$ et la comparaison entre $t[0]$ et $t[1]$ initialise correctement le min et le max de $t[0 \dots 1]$. Dans les deux cas, $\mathcal{I}(i)$ est vérifiée.

— **Conservation** : supposons $\mathcal{I}(i)$ vraie. Lors de l'itération, on considère la paire $\{t[i], t[i+1]\}$. Soit $a = \min(t[i], t[i+1])$ et $b = \max(t[i], t[i+1])$. Le minimum de $t[0 \dots i+1]$ est $\min(*\texttt{pmin}, a)$ et son maximum est $\max(*\texttt{pmax}, b)$. Après les deux tests et mises à jour, puis l'incrémentation $i \leftarrow i+2$, la propriété $\mathcal{I}(i+2)$ est vérifiée.

— **Sortie** : la boucle se termine quand tous les éléments ont été traités ($i = n$). Alors $\mathcal{I}(n)$ garantit que `*pmin` et `*pmax` sont bien le minimum et le maximum du tableau complet $t[0 \dots n-1]$.